



Code Study Guide — Phase 1 Backend
Every script explained line by line, in plain English

17 files · Python 3.12 · FastAPI · Supabase · April 2026

Table of Contents

FOUNDATION

- 2. main.py — The server entry point
- 3. requirements.txt — Every library explained

MODELS (DATA BLUEPRINTS)

- 5. app/models/task.py — TaskBubble & Cluster
- 6. app/models/finance.py — FinanceEntry, Budget, Insight
- 7. app/models/user.py — UserProfile & Preferences

DATABASE LAYER

- 9. app/db/client.py — Supabase connection
- 10. app/db/tasks.py — Task database queries
- 11. app/db/finance.py — Finance database queries

ROUTERS (HTTP ENDPOINTS)

- 13. app/routers/tasks.py — Task API endpoints
- 14. app/routers/finance.py — Finance API endpoints

SERVICES (BUSINESS LOGIC)

- 16. app/services/scoring.py — Pressure score calculator
- 17. app/services/ai_router.py — AI provider gateway
- 18. app/services/finance_categorizer.py — Merchant rules

TESTS

- 20. tests/test_scoring.py — Automated scoring tests

main.py

main.py

ENTRY POINT

The front door of the entire backend. Python runs this file first when you start the server.

REAL-WORLD ANALOGY

Think of `main.py` as the reception desk of a large office building. It doesn't do any actual work itself — it just opens the doors, directs visitors to the right department (router), and prints a message when the building opens and closes for the day.

The imports — what gets brought in

```
from contextlib import asynccontextmanager
```

STANDARD LIBRARY IMPORT

contextlib is a built-in Python module (it comes with Python itself, no installation needed). **asynccontextmanager** is a decorator that lets you write a function that runs code at startup and at shutdown. The `async` part means it works with Python's asynchronous system. You use it here to print a message when the server starts and another when it stops.

```
from dotenv import load_dotenv
```

THIRD-PARTY LIBRARY (PYTHON-DOTENV)

dotenv is a library that reads your `.env` file and loads every line in it (like `SUPABASE_URL=https://...`) into Python's environment variables. After calling `load_dotenv()`, your code can access those values anywhere using `os.environ.get("SUPABASE_URL")`. Without this, your secret keys would never be loaded and every database call would fail.

WHY LOAD_DOTENV() IS CALLED BEFORE ANYTHING ELSE

Notice that `load_dotenv()` is called on line 7, before any other app imports. This is intentional. When Python imports `app.db.client`, that file immediately reads `SUPABASE_URL` from the environment. If the `.env` file hasn't been loaded yet, that value would be empty and the database connection would fail. Order matters here.

```
from fastapi import FastAPI
```

THIRD-PARTY LIBRARY (FASTAPI)

FastAPI is the web framework — the entire backbone of the backend. A web framework is a tool that handles all the boring parts of running a web server: receiving HTTP requests, routing them

to the right function, validating the data, and sending back responses. You create one FastAPI instance and it becomes "the app".

```
from fastapi.middleware.cors import CORSMiddleware
```

PART OF FASTAPI

CORS stands for Cross-Origin Resource Sharing. Browsers and mobile apps have a security rule: they won't make requests to a server on a different domain unless that server explicitly allows it. **CORSMiddleware** adds a permission header to every response saying "yes, requests from other origins are allowed". Without this, the React Native phone app would be blocked from talking to this backend.

```
from app.routers import tasks, finance, users
```

LOCAL IMPORT (OUR OWN CODE)

This imports the three router files from the `app/routers/` folder. Each router is a collection of related endpoints. Importing them here and then calling `app.include_router()` is how FastAPI learns about all the URLs your server will respond to.

The lifespan function

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    print("Orbi API is running")
    yield
    print("Orbi API is shutting down")
```

The `@asynccontextmanager` decorator turns this function into a lifecycle manager. Everything **before** the `yield` runs when the server starts. Everything **after** the `yield` runs when the server shuts down. The `yield` keyword is the pause in the middle — "keep the server running until someone stops it". In Phase 2 this is where you'd connect to the database on startup and disconnect cleanly on shutdown.

Creating the app

```
app = FastAPI(
    title="Orbi API",
    version="0.1.0",
    lifespan=lifespan,
)
```

This creates the single FastAPI application instance. The title and version appear in the automatically generated documentation at /docs. The lifespan=lifespan tells FastAPI to use your lifecycle function for startup/shutdown.

Adding CORS middleware

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],    # TODO: restrict before production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Middleware wraps around every single request your server receives. `allow_origins=["*"]` means "accept requests from any website or app". The `*` is a wildcard meaning "everything". Before going to production this should be tightened to only allow requests from the Orbi app, but during development the wildcard is convenient.

Registering the routers

```
app.include_router(tasks.router, prefix="/api/v1")
app.include_router(finance.router, prefix="/api/v1")
app.include_router(users.router, prefix="/api/v1")
```

Each router defines its own sub-prefix (e.g. /tasks or /finance). Adding /api/v1 here means every URL will be /api/v1/tasks, /api/v1/finance, etc. The v1 is a version number — if you ever break the API, you can create v2 endpoints without removing v1, so old clients don't break.

The health endpoint

```
@app.get("/health")
async def health():
    return {"status": "ok", "app": "Orbi"}
```

A health endpoint is a simple URL (GET /health) that returns a quick confirmation the server is alive. Monitoring systems, load balancers, and deployment tools ping this URL to know the server is healthy. It doesn't touch the database — it just returns immediately. You tested this when you first started the server.

`requirements.txt`

requirements.txt

DEPENDENCIES

The shopping list of every external library the project needs. Run `pip install -r requirements.txt` to install them all.

REAL-WORLD ANALOGY

Python doesn't come with everything built in. Think of `requirements.txt` like a recipe's ingredient list. You need these specific ingredients (libraries) before you can cook the dish (run the app). The version numbers (e.g. `fastapi==0.115.0`) ensure everyone uses the exact same version — like specifying "Tesco own-brand flour" rather than just "flour".

Web Framework

`fastapi==0.115.0`

The web framework. Creates all HTTP endpoints, handles routing, validates request data automatically, and generates the `/docs` interactive test page. FastAPI is asynchronous by default — it can handle many simultaneous users without slowing down, because while waiting for a database response it can serve other requests.

`uvicorn[standard]==0.30.0`

FastAPI creates the app logic, but something must actually listen on a network port for incoming connections. Uvicorn is that server. Think of FastAPI as the engine and Uvicorn as the car body around it. The `[standard]` part installs optional extras for better performance and WebSocket support (used later for real-time bubble sync).

Database

`supabase==2.5.0`

The official Python client for Supabase. Without this, talking to your Supabase database would require writing raw HTTP requests manually — error-prone and verbose. This library wraps all that complexity into clean Python calls like `supabase.table("task_bubbles").select("*").execute()`.

`psycopg2-binary==2.9.11`

The low-level connection driver that actually opens a socket to PostgreSQL. Supabase uses PostgreSQL under the hood. The `-binary` suffix means it comes pre-compiled — no additional C

compiler needed on your machine. Other libraries (like Supabase) use this internally without you having to touch it directly.

pgvector==0.3.2

Adds support for storing and searching AI embeddings (vectors) in PostgreSQL. An embedding is a list of numbers that represents the "meaning" of a piece of text. When you search "that car insurance thing", pgvector finds the most semantically similar items in the database. Used in Phase 2 onwards.

Data Validation

pydantic==2.12.5

When the phone app sends data to your backend, Pydantic automatically checks that every field is the right type and within allowed ranges. If the phone sends a string where a number is expected, Pydantic rejects it before your code even sees it — with a clear error message. All the classes in `app/models/` are Pydantic models.

pydantic-settings==2.13.1

An extension of Pydantic for configuration. Instead of calling `os.environ.get("SUPABASE_URL")` everywhere, you define a `Settings` class and Pydantic-settings automatically reads all environment variables into it with type checking and autocomplete.

python-dotenv==1.0.1

Reads the `.env` file and loads each line into Python's environment variables. Must be called before any other import tries to read a secret key. Already used in `main.py` on line 7.

AI Providers

groq==0.9.0

Official Python client for the Groq API. Groq runs Llama 3.1 8B on specialised hardware that makes inference extremely fast and cheap (~\$0.05 per million tokens). Used for Free and Pro tier users. The client abstracts all the HTTP requests into one function call.

anthropic==0.34.0

Official Python client for the Anthropic API (Claude). Used only for Premium tier users who pay £9.99/month. Claude Sonnet 4.6 is more powerful than Llama 8B for complex reasoning and long memory synthesis, but costs significantly more. The `ai_router.py` decides which client to call.

Security & Auth

`python-jose[cryptography]==3.3.0`

Reads and verifies JWT tokens. When a user logs in via Supabase, they receive a JWT (JSON Web Token) — a cryptographically signed string that proves their identity. Every protected endpoint receives this token in the Authorization header. This library decodes it and extracts the user's ID. Without this, anyone could claim to be any user.

`passlib[bcrypt]==1.7.4`

Password hashing. If you ever store a password, you never store the raw text — you store a bcrypt hash (a scrambled, one-way representation). Even if the database is compromised, the attacker cannot reverse the hash back to the password. This is a safety net for any custom auth flows beyond Supabase.

`httpx==0.27.0`

An async-native HTTP client for making requests to external services. Python's built-in `urllib` is not async — it blocks the server while waiting. `httpx` is used internally by the Supabase and Groq clients and for any outbound HTTP calls the backend makes.

Testing

`pytest==8.2.0`

The testing framework. You write functions that check whether your code produces the right output, then run `pytest tests/` and it finds and runs all of them, reporting which passed and which failed. Professional codebases run these before every deployment to catch regressions.

`pytest-asyncio==0.23.7`

Since all Orbi code uses `async/await`, the tests must be `async` too. This plugin extends `pytest` to understand `async` test functions. Without it, `async` tests would appear to pass but would silently skip all their assertions — a dangerous false positive.

`app/models/task.py`

app/models/task.py

MODEL

Defines the exact shape and rules for TaskBubble data. The blueprint every other part of the code follows.

REAL-WORLD ANALOGY

A model is like a government form. It specifies every field that must exist ("full name — required"), what type of value each field takes ("date of birth — must be a date, not text"), and which fields are optional. If you hand in a form with the wrong type of data in a field, it's rejected before it's processed.

Imports

```
from datetime import datetime
```

Python's built-in date/time type. Used for fields like `created_at` and `due_at` that store a specific point in time.

```
from enum import Enum
```

Enum (Enumeration) is a way to define a fixed set of allowed values for a field. Instead of storing a status as a free-text string where a typo like "complted" could sneak in, you define an Enum: only "active", "completed", "snoozed", or "archived" are valid. Any other value is automatically rejected.

```
from typing import Optional
```

`Optional[str]` means a field can be either a string or `None` (missing/not provided). For example, `description` is optional — a bubble can exist without a description.

```
from uuid import UUID
```

A UUID (Universally Unique Identifier) is a 128-bit number formatted as `550e8400-e29b-41d4-a716-446655440000`. Every database record uses a UUID as its ID instead of a simple integer (1, 2, 3...) because UUIDs are guaranteed unique across all servers and clients — no coordination needed.

```
from pydantic import BaseModel, Field
```

BaseModel is the Pydantic base class — every model inherits from it to get automatic validation. **Field** lets you add extra constraints to a field, like `ge=1` (greater-than-or-equal to 1) and `le=10` (less-than-or-equal to 10) for the importance field.

class TaskStatus(str, Enum)

```
class TaskStatus(str, Enum):  
    active = "active"  
    completed = "completed"  
    snoozed = "snoozed"  
    archived = "archived"
```

By inheriting from both `str` and `Enum`, each value is simultaneously a string and an enum member. This means when it's stored in the database or sent in JSON, it appears as a plain string ("active") — not some weird Python object. But in code, you reference it as `TaskStatus.active`, which prevents typos.

active

The default state. The bubble is visible in the universe and being tracked.

completed

The user has finished this task. The bubble collapses/disappears from the universe view.

snoozed

The user has dismissed the bubble temporarily. It will reappear later.

archived

Soft-deleted. The task is hidden from the user but kept in the database for memory synthesis and audit purposes. The app never hard-deletes a task.

class Visibility(str, Enum)

Defines who can see a bubble. Added from day one even though sharing isn't built yet — doing it now prevents a painful database migration later.

private

Default. Only the owner can see this bubble.

shared

Another user can view this bubble (read-only). Reserved for Phase 5+.

collaborative

Another user can edit and contribute to this bubble. Reserved for Phase 5+.

class TaskBubble(BaseModel) — The complete bubble

This is the full database representation of a bubble. Every field here corresponds to a column in the `task_bubbles` table in Supabase.

id: UUID

The bubble's unique identifier. Generated automatically (`uuid4()`) when the bubble is created.

owner_id: UUID

The `user_id` of whoever created this bubble. Every query filters by `owner_id` so users only ever see their own bubbles. Never accepted from the request body — always taken from the auth token to prevent one user from claiming another's data.

title: str

The main text of the bubble — "Pay rent", "Call dentist", etc.

description: Optional[str] = None

Extra context. The `= None` means if the phone app doesn't send this field, it defaults to `None` rather than causing an error.

status: TaskStatus

Must be one of the four values in `TaskStatus`. Pydantic rejects anything else.

```
due_at: Optional[datetime] = None
```

When the task is due. Optional because not all tasks have a deadline. Used by the scoring service to calculate `deadline_weight`.

```
importance: int = Field(ge=1, le=10)
```

How important this task is on a 1–10 scale. The `Field(ge=1, le=10)` constraint means Pydantic will reject a value of 0 or 11. Drives the `importance_weight` in the pressure score.

```
urgency_score: float
```

A numeric score 0–10 representing urgency separate from deadline proximity. Initially 0.0 — will be enriched by the AI agent in Phase 2.

```
pressure_score: float
```

The computed bubble pressure, 0–10. Higher = bigger, brighter bubble in the universe. Calculated by `scoring.py` whenever the task is created or updated.

```
domain_hint: Optional[str] = None
```

A label like "Finance", "Health", "Work" that suggests which cluster this bubble belongs to. Set by the AI task parser from the user's speech.

```
parent_cluster_id: Optional[UUID] = None
```

The UUID of the Cluster this bubble belongs to, if any. None until the cluster manager assigns it.

```
source_type: str
```

How this bubble was created. "voice" = user spoke it. "manual" = user typed it. "import" = brought in from another system. Used for analytics.

```
confidence: float = Field(ge=0.0, le=1.0)
```

How confident the AI was in parsing this bubble (0.0 to 1.0). If confidence is low (e.g. 0.4), the app flags the bubble for user confirmation before saving. Manual entries have confidence 1.0.

```
visibility: Visibility = Visibility.private
```

Who can see this bubble. Defaults to private. The `= Visibility.private` sets the default so you don't have to specify it every time.

```
created_at / updated_at: datetime
```

Timestamps for when the bubble was first created and last modified. `updated_at` is used by the `attention_decay` function in scoring — if it's been weeks since this was touched, the pressure score rises.

class TaskBubbleCreate(BaseModel) — What the phone sends

When the phone app wants to create a new bubble, it sends this shape — not a full `TaskBubble`. Notice it has no `id`, no `created_at`, no `updated_at` — those are generated server-side. This prevents the client from injecting their own IDs or timestamps.

class TaskBubbleUpdate(BaseModel) — Partial edits

Every field here is `Optional` with a default of `None`. This means the phone can send just the fields it wants to change — you don't have to resend the entire bubble just to update the title. Fields that are `None` are excluded from the database update.

class Cluster(BaseModel) — A group of related bubbles

```
name: str
```

The cluster name, e.g. "Finance", "Health", "Work". Displayed as a label in the universe.

```
weight_score: float
```

The aggregate pressure of all bubbles in this cluster. A cluster with many urgent bubbles gets a high `weight_score` and expands visually.

```
active_count: int
```

How many non-archived bubbles are currently in this cluster.

```
color: str
```

The hex colour used to render this cluster in the universe (e.g. "#7c6fe0").

app/models/finance.py

app/models/finance.py

MODEL

Blueprints for all finance-related data: expenses, budgets, and AI insights.

```
from datetime import date, datetime
```

Note that two different types are imported: `date` (just the calendar date, e.g. 2026-04-18) and `datetime` (a specific moment in time, e.g. 2026-04-18 14:32:00). Finance entries use `date` for `entry_date` because a purchase happened on a day — the exact time doesn't matter. But `created_at` uses `datetime` because we want to know the exact moment the record was saved.

class EntryType(str, Enum)

Only two valid values: "income" (money coming in) or "expense" (money going out). Pydantic rejects anything else. This enum prevents a category of bugs where mistyped values slip into the database.

class InsightSeverity(str, Enum)

Three levels for AI-generated financial observations:

info

General observation. "You spent £200 on dining this month." Not urgent.

warning

Something worth paying attention to. "Food budget is 80% used with 12 days left."

alert

Needs immediate attention. "This £800 Amazon charge is 3x your usual spend." Likely triggers a push notification.

class FinanceEntry(BaseModel) — One transaction

amount: float

The transaction amount. Always positive — the direction (in or out) is tracked by `entry_type`, not by making this negative.


```
currency: str = "GBP"
```

Defaults to GBP since Orbi is UK-first. The default means the phone doesn't have to send this field unless the user is transacting in a different currency.

```
merchant: str
```

The raw merchant name as the user typed it or as it appeared on the bank statement. e.g. "TESCO STORES 3289". The categorizer normalises this.

```
category: str
```

Set by the finance categorizer — "groceries", "transport", "dining", etc. Not free-text from the user.

```
linked_bubble_id: Optional[UUID] = None
```

If this expense is related to a specific TaskBubble (e.g. a dentist appointment linked to a "Book dentist" task), this stores that connection. Optional — most expenses aren't task-linked.

class FinanceBudget(BaseModel) — A monthly spending limit

```
monthly_limit: float
```

The maximum amount the user wants to spend in this category per month. e.g. £200 for groceries.

```
alert_threshold: float = Field(default=0.8, ge=0.0, le=1.0)
```

A fraction of monthly_limit at which to trigger an alert. Default 0.8 = alert when 80% of budget is spent. The `ge=0.0`, `le=1.0` constraint ensures this is always between 0 and 1 (a valid fraction).

```
current_spend: float
```

Running total of what's been spent in this category this month. Updated each time a matching expense is logged.

class FinanceInsight(BaseModel) — An AI observation

```
insight_text: str
```

The human-readable observation generated by the `finance_agent`. e.g. "You spent 40% more on dining this month compared to last month."

severity: `InsightSeverity`

How important this insight is — info, warning, or alert. Controls how prominently the app surfaces it.

`app/models/user.py`

app/models/user.py

MODEL

The user's identity, subscription tier, and personal preferences.

```
from pydantic import EmailStr
```

EmailStr is a special Pydantic type that validates email addresses. If a field is typed as EmailStr, Pydantic checks that the value actually looks like an email address (has an @ sign, a domain, etc.). It doesn't just accept any string.

class SubscriptionTier(str, Enum)

The most important field in the entire system — it controls which AI the user gets.

free

Groq Llama 8B. Limited bubbles and clusters. Rule-based insights only. ~\$0.001/user/day in AI costs.

pro

Groq Llama 8B. Unlimited bubbles. Full voice debrief. Finance intelligence. ~\$0.50–1/user/month.

premium

Claude Sonnet 4.6. Deep reasoning, long memory, multi-step agent tasks. ~\$3–5/user/month.

CRITICAL SECURITY RULE

The subscription tier is ALWAYS verified server-side by reading from the database. It is never trusted from the client request. A user cannot upgrade themselves by sending a modified request — the server always checks the real value in the database.

class UserPreference(BaseModel)

```
quiet_hours_start / quiet_hours_end: time
```

Python's time type (just hours and minutes, no date). Defines when the app should not send notifications. e.g. 22:00 to 08:00 = no notifications at night.

```
proactivity_level: int
```

1 to 5 scale. Level 1 = only notify when the user asks. Level 5 = the assistant proactively surfaces insights, suggests tasks, and nudges you throughout the day.

```
preferred_reminder_channel: str
```

"push" (phone notification), "voice" (spoken reminder), or "in-app". Controls how the assistant delivers reminders.

```
model_config = {"json_schema_extra": {"examples": [...]}}
```

This adds an example to the auto-generated API documentation at /docs. When a developer opens the interactive docs, they see a pre-filled example request. It has no effect on how the code runs — it's purely for documentation.

app/db/client.py

app/db/client.py

DATABASE

Opens and manages the single connection to Supabase. Every other db file imports from here.

REAL-WORLD ANALOGY

Think of this as the office's one phone line to headquarters. You don't open a new phone line every time someone needs to make a call — you share the one existing line. This file creates that one shared connection and hands it out to anyone who asks.

```
import logging
```

Python's built-in logging system. Instead of using `print()` for debugging messages (which you'd have to remove before production), logging lets you write structured log messages that can be filtered by severity, routed to files, or sent to monitoring services.

```
logger = logging.getLogger(__name__)
```

Creates a logger named after this file (`app.db.client`). Each module has its own named logger so you can filter logs by where they came from. `__name__` is a Python built-in that automatically equals the current module's name.

```
_client: Client | None = None
```

A module-level variable that starts as `None`. The underscore prefix `_` is a Python convention meaning "this is private — don't import it directly from outside this module". `Client | None` means it can be either a Supabase Client object or `None`.

def get_client() → Client

This is a **lazy singleton**. "Singleton" means only one instance ever exists. "Lazy" means it's only created when first needed, not at import time.

```
def get_client() -> Client:
    global _client
    if _client is None:
        url = os.environ.get("SUPABASE_URL", "")
        key = os.environ.get("SUPABASE_SERVICE_KEY", "")
        _client = create_client(url, key)
    return _client
```

First call: `_client` is `None`, so it reads the credentials from the environment and creates the connection. Every subsequent call: `_client` is already set, so it returns the existing connection immediately. This avoids opening hundreds of database connections — one is enough.

`SUPABASE_SERVICE_KEY` (not `SUPABASE_ANON_KEY`)

Supabase has two keys: the anon key (used on the client/phone app) and the service key (used on the server). The service key bypasses Row Level Security — it can read and write any row for any user. This is necessary for the backend to serve all users. This key must NEVER be sent to the phone app.

`async def test_connection() → bool`

Runs a lightweight query (`SELECT id FROM user_profiles LIMIT 1`) to check the database is reachable. Returns `True` if successful, `False` if anything fails. Used at startup for diagnostics. The `try/except` block means any error is caught, logged, and returns `False` instead of crashing the whole server.

app/db/tasks.py

app/db/tasks.py

DATABASE

All database queries for TaskBubble. No business logic — only reading and writing to Supabase.

REAL-WORLD ANALOGY

This is the filing clerk. It has one job: fetch records from the cabinet, or file new ones. It doesn't make decisions about what to do with records — that's the service layer's job. Give it a `user_id` and it returns their tasks. Give it task data and it saves it.

async def fetch_tasks_for_user(user_id)

```
response = (
    get_client().table("task_bubbles")
    .select("*")
    .eq("owner_id", str(user_id))
    .neq("status", "archived")
    .order("pressure_score", desc=True)
    .execute()
)
```

This is a fluent query builder — you chain method calls to build up a SQL query. In plain SQL this would be: `SELECT * FROM task_bubbles WHERE owner_id = '...' AND status != 'archived' ORDER BY pressure_score DESC`. Each method adds one clause:

- `.table("task_bubbles")` — target this database table
- `.select("*")` — fetch all columns
- `.eq("owner_id", ...)` — WHERE owner_id equals this value
- `.neq("status", "archived")` — AND status is not equal to "archived"
- `.order("pressure_score", desc=True)` — ORDER BY pressure_score descending (highest first)
- `.execute()` — actually run the query

async def fetch_task_by_id(task_id, owner_id)

Uses `.single()` at the end — this tells Supabase to expect exactly one result and return it as a dict rather than a list. The `owner_id` filter is critical: it ensures a user can only fetch their own task. Without it, knowing another task's UUID would let any user read it.

async def insert_task(payload: dict)

Takes a Python dictionary and inserts it as a new row. `response.data[0]` returns the first (and only) inserted row, including any database-generated values like auto-timestamps. The `[0]` indexing extracts from the list since Supabase always returns a list even for single inserts.

async def update_task(task_id, owner_id, payload: dict)

Sends only the changed fields to the database using `.update(payload)`. Again filtered by both `task_id` and `owner_id` — ownership is verified at the database level, not just in application code. Belt-and-suspenders security.

async def archive_task(task_id, owner_id)

Sets `status = "archived"` rather than deleting the row. This is called a **soft delete**. The data is preserved for memory synthesis, undo functionality, and audit trails. The task simply disappears from the user's view because every fetch filters out archived tasks.

app/db/finance.py

app/db/finance.py

DATABASE

Database queries for finance entries and budgets, including date-range filtering.

async def fetch_entries_for_user(user_id, category, month)

This function supports optional filters. If `category` is provided, it adds a `WHERE category = ...` clause. If `month` is provided (in "YYYY-MM" format), it adds a date range filter. Both can be combined.

Month range filtering

```
query = query.gte("entry_date", f"{month}-01").lt(
    "entry_date", _next_month(month)
)
```

`.gte` = "greater than or equal to" ($\geq$). `.lt` = "less than" ($<$). So for month "2026-04", this becomes: `entry_date  $\geq$  2026-04-01 AND entry_date  $<$  2026-05-01`. This captures the entire month without worrying about the last day (April has 30 days, not 31 or 28).

async def upsert_budget(payload)

Upsert = "update or insert". If a budget for this user+category already exists, update it. If it doesn't exist, insert it. The `on_conflict="user_id,category"` tells Supabase which combination of fields constitutes a "duplicate". This means the phone app can call `POST /finance/budgets` without first checking whether a budget exists — the server handles it idempotently.

def _next_month(month: str)

A private helper function (underscore prefix). Given "2026-04", returns "2026-05-01". Handles December correctly (December + 1 = January of next year). Used to build the exclusive upper bound for month range queries.

`app/routers/tasks.py`

app/routers/tasks.py

ROUTER

Defines the HTTP endpoints for TaskBubble. Receives requests, validates auth, delegates to services and db, returns responses.

REAL-WORLD ANALOGY

The router is a receptionist. When someone calls the office (HTTP request arrives), the receptionist checks their ID (auth header), decides which department to forward them to (service/db layer), and relays the answer back. The receptionist doesn't solve problems themselves.

```
from fastapi import APIRouter, Depends, Header, HTTPException, status
```

- **APIRouter** — creates a sub-router with its own prefix (/tasks). You group related endpoints into one router.
- **Depends** — FastAPI's dependency injection system. `Depends(get_current_user)` means "call `get_current_user` first and pass its return value into this endpoint". Used for auth.
- **Header** — tells FastAPI to extract a value from the HTTP request headers (not the body).
- **HTTPException** — raises an HTTP error response (like 404 Not Found) and stops execution.
- **status** — a module of HTTP status code constants. `status.HTTP_201_CREATED` is cleaner and self-documenting compared to the bare number 201.

```
router = APIRouter(prefix="/tasks", tags=["tasks"])
```

Creates the router. `prefix="/tasks"` means all endpoints in this file start with /tasks. Combined with main.py's `prefix="/api/v1"`, every URL becomes `/api/v1/tasks/...` `tags=["tasks"]` groups these endpoints together in the /docs page.

`async def get_current_user(authorization: str = Header(...))`

This is the **auth dependency**. `Header(...)` extracts the "Authorization" HTTP header from every request. The `...` (Ellipsis) means this header is required — if it's missing, FastAPI automatically returns a 422 error.

PLACEHOLDER — NOT REAL AUTH YET

Currently this function returns a hardcoded UUID (`00000000-0000-0000-0000-000000000001`) for any Bearer token. This is a placeholder so all endpoints can be tested before real JWT verification is wired in Phase 2. In production, this must decode the Supabase JWT and extract the real `user_id`.

def _error(message, error_code)

A tiny private helper that builds a consistent error response dict. Every error from this backend has the same shape: {"message": "...", "error_code": "..."}.

This makes it easy for the phone app to handle errors programmatically (check the error_code) while showing the message to the user.

GET /api/v1/tasks — list_tasks

```
@router.get("", response_model=list[TaskBubble])
async def list_tasks(user_id: UUID = Depends(get_current_user)):
    rows = await tasks_db.fetch_tasks_for_user(user_id)
    return rows
```

Returns all non-archived tasks for the authenticated user, ordered by pressure_score descending.

response_model=list[TaskBubble] tells FastAPI to validate and serialize the response using the TaskBubble model — it strips any extra fields that shouldn't go to the client.

POST /api/v1/tasks — create_task

The most complex endpoint. Here's what happens step by step:

1. Auth is verified. user_id comes from the token, not the body.
2. A full TaskBubble object is constructed with a new UUID and timestamps.
3. calculate_pressure_score(task) computes the initial pressure score.
4. task.model_dump(mode="json") converts the Pydantic object to a plain Python dict, converting UUID objects to strings and datetimes to ISO format strings (ready for JSON/database).
5. The dict is inserted into the database.
6. The saved row (returned from the DB) is sent back as the response.

PATCH /api/v1/tasks/{task_id} — update_task

The {task_id} in the URL is a path parameter — FastAPI automatically extracts it and passes it to the function. The endpoint first fetches the existing task, then merges the incoming changes on top using dict unpacking: {**existing, **body.model_dump(exclude_none=True)}. The exclude_none=True means only fields actually sent in the request are included — None fields are skipped.

DELETE /api/v1/tasks/{task_id} — delete_task

Returns HTTP 204 No Content on success (no response body — there's nothing to return). Calls archive_task, not an actual delete. If the task isn't found (or belongs to another user), returns 404.

`app/routers/finance.py`

app/routers/finance.py

ROUTER

Endpoints for logging expenses, listing entries, viewing spending summaries, and managing budgets.

```
from collections import defaultdict
```

`defaultdict` is a special dictionary that automatically creates a default value for missing keys. Here it's used with `float`, meaning any category that hasn't been seen before automatically gets a value of `0.0`. Without this, accessing a key that doesn't exist in a normal dict would raise a `KeyError`.

```
from fastapi import Query
```

`Query` is used to define optional URL query parameters — values passed after a `?` in the URL. For example: `GET /finance/entries?category=groceries&month=2026-04`. The `Query(default=None)` makes them optional.

POST /api/v1/finance/entries — create_entry

The key step here is automatic categorization before saving:

```
category = categorize_merchant(body.merchant)
```

Before any database write, the merchant name is run through the rule-based categorizer. If "Tesco" is in the merchant name, `category` becomes "groceries" automatically. The user doesn't have to categorize anything themselves for known merchants.

GET /api/v1/finance/summary — get_summary

This endpoint does the aggregation in Python rather than in SQL. It fetches all entries for the month, then loops through them:

- Income entries are added to `total_income`
- Expense entries are added to `total_spend` and to `totals[category]`

The result is a breakdown like: `{"groceries": 145.50, "dining": 88.00, ...}` plus overall totals. The `round(..., 2)` calls ensure no floating-point errors produce values like `145.500000001`.

POST /api/v1/finance/budgets — upsert_budget

Returns HTTP 200 (not 201) because an upsert might update an existing record rather than create a new one. HTTP 201 specifically means "created new" — 200 is the correct code for "created or updated".

`app/services/scoring.py`

app/services/scoring.py

SERVICE

Computes the pressure score (0–10) for every bubble. Pure maths — no AI, no database calls, no side effects.

REAL-WORLD ANALOGY

Think of this as a calculator that measures "how loud a bubble is shouting for your attention". It takes several factors — how close the deadline is, how important the task is, whether other things depend on it, and how long you've been ignoring it — and combines them into one number from 0 to 10.

def _deadline_weight(due_at)

A stepped scale that accelerates as the deadline approaches. The steps are designed so the bubble visually grows as urgency increases:

```
days_remaining = (due_at - now).total_seconds() / 86400
```

Subtracting two datetime objects gives a `timedelta` (a duration). `.total_seconds()` converts it to seconds. Dividing by 86400 (seconds in a day) gives days. Negative values mean the task is overdue.

- Overdue → 10.0 (maximum urgency)
- ≤ 1 day → 8.0
- ≤ 3 days → 5.0
- ≤ 7 days → 2.0
- More than 7 days → 0.0 (not yet urgent)

def _importance_weight(importance)

Maps the 1–10 importance value to a 0.5–5.0 contribution. Dividing by 2 (not using raw importance) prevents importance alone from dominating the total score. A task marked "10 importance" contributes 5 points — half the maximum possible score, which leaves room for other factors.

def _dependency_weight(dependency_count)

If other people or tasks are blocked waiting for this one, it should feel more urgent. Even one dependency is a significant cost-of-delay increase, so the jump from 0→1 dependency is intentionally large (0→1.0). Three or more dependencies cap at 2.0.

def _attention_decay(updated_at)

Without this, tasks with no deadline and no dependencies would sit at a low score forever — quietly forgotten. This function checks the last time the task was touched:

- Untouched for more than 14 days → +3.0 (significant revival boost)
- Untouched for more than 7 days → +1.5 (gentle nudge)
- Recently updated → 0.0 (no boost)

def calculate_pressure_score(task, dependency_count=0)

The main function. Adds all four component weights together, then clamps the result to the [0, 10] range using `min(max(raw_score, 0.0), 10.0)`. The inner `max(raw_score, 0.0)` ensures it's never negative. The outer `min(..., 10.0)` ensures it never exceeds 10. Then `round(..., 4)` gives 4 decimal places — enough precision for the bubble size renderer.

Why no AI in scoring?

Pressure scoring is deterministic — the same task always gets the same score. This means it's fast (no network call), free (no API cost), testable (you can write exact assertions), and predictable for the user. AI is only used for things that require understanding language or generating natural text — not for mathematical calculations.

`app/services/ai_router.py`

app/services/ai_router.py

SERVICE

The single gateway for every AI call in the backend. Routes to Groq or Claude based on the user's subscription tier.

REAL-WORLD ANALOGY

Think of this as a telephone switchboard operator. When a call comes in asking for "an AI response", this file checks who's calling (their subscription tier) and puts them through to the right expert: the budget-friendly Llama for free/pro users, or the premium Claude specialist for premium users. All other code just dials the switchboard — they never call Groq or Claude directly.

`import time`

Python's built-in time module. Used here for `time.monotonic()` — a high-resolution clock that measures elapsed time. It's used to measure how many milliseconds an AI call takes, which is logged for performance monitoring.

`from pathlib import Path`

`Path` is Python's modern way to work with file system paths. `Path(__file__)` gets the path of the current file. `.parent` goes up one directory. Building `_PROMPTS_DIR` this way works on any operating system without hardcoding slashes.

`_groq_client = Groq(api_key=os.environ["GROQ_API_KEY"])`

Creates the Groq client at module load time (when the file is first imported). Note `os.environ["GROQ_API_KEY"]` with square brackets — this raises a `KeyError` if the key is missing (unlike `.get()` which would silently return `None`). Failing loud at startup is better than failing mysteriously later.

`_FALLBACK_RESPONSE`

A constant string returned when an AI call fails. Instead of crashing the entire request, the user gets a gentle "try again" message. The real error is logged server-side for debugging. This is called a **graceful degradation** pattern.

`def load_prompt(agent_name, version=1)`

Reads a prompt file from `app/prompts/` and returns it as a plain string. The naming convention is `{agent_name}_v{version}.md`. By storing prompts in separate files rather than hardcoded Python strings, you can improve the AI's behaviour by editing a text file without touching any Python code. The `version` parameter means you can deploy a new prompt version (v2) without deleting the old one (v1).

`async def get_ai_response(prompt, user_tier, system_prompt, max_tokens=500)`

The public interface. Every agent calls this function — none call Groq or Claude directly. This is the **single point of control** for all AI spending in the entire backend.



`async def _call_groq(prompt, system_prompt, max_tokens)`

```

response = _groq_client.chat.completions.create(
    model=_GROQ_MODEL,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": prompt},
    ],
    max_tokens=max_tokens,
)
  
```

The `messages` list follows the chat format used by all modern AI APIs. `system` = the AI's instructions (loaded from the prompt file). `user` = what the user actually said. The AI responds in the "assistant" role (not shown here — that's in the response).

After the call, the function logs: which provider, which model, how many tokens were used, and how many milliseconds it took. This data is essential for billing awareness and performance monitoring.

`async def _call_claude(prompt, system_prompt, max_tokens)`

Same pattern as Groq but using the Anthropic client. One key difference: Anthropic's API separates `system` from the `messages` list, while Groq puts `system` inside `messages`. Both achieve the same result — the differences are just API design choices by each company.

WHY WRAP EVERYTHING IN TRY/EXCEPT?

External API calls can fail for many reasons: network timeout, provider outage, rate limit hit, invalid API key. Without `try/except`, any of these would crash your entire request with an unhelpful 500 error. With it, the user gets a friendly "try again" message and your server logs the real error for investigation.

`app/services/finance_categorizer.py`

app/services/finance_categorizer.py

SERVICE

Sorts expenses into categories using a lookup table of known merchants — zero AI, zero cost.

REAL-WORLD ANALOGY

Imagine a human bookkeeper who has memorised that anything from Tesco goes in the "Groceries" column and anything from Netflix goes in "Subscriptions". They don't need to think about it — they just check their mental list. Only when they encounter a merchant they've never seen ("Priya's Boutique") do they need to ask someone for help (the AI).

_RULES — the lookup table

```
_RULES: list[tuple[str, str]] = [  
    ("tesco", "groceries"),  
    ("amazon prime", "subscriptions"),  
    ("amazon", "shopping"), # after "amazon prime"  
    ...  
]
```

A list of tuples. Each tuple is a pair: (keyword to match, category to assign). The list is ordered — this matters. "amazon prime" appears before "amazon" so that "Amazon Prime Video" is correctly categorised as a subscription, not shopping. If the order were reversed, "amazon prime" would match the bare "amazon" rule first and be wrongly categorised as shopping.

def categorize_merchant(merchant: str) → str

```
normalised = merchant.lower().strip()  
for keyword, category in _RULES:  
    if keyword in normalised:  
        return category  
return "uncategorized"
```

- `.lower()` — converts to lowercase so "TESCO STORES" matches "tesco"
- `.strip()` — removes leading/trailing whitespace that might come from bank imports
- `keyword in normalised` — checks if the keyword appears anywhere in the merchant name (substring match, not exact match)
- First match wins — the function returns immediately on the first matching rule
- If no rule matches → returns "uncategorized" — the caller can optionally escalate to the AI

WHY RULE-BASED FIRST?

The vast majority of everyday spending is at known merchants. Running AI on "Tesco £45" would waste money and add 200–500ms latency for a question that can be answered in microseconds. AI is only called for the small percentage of genuinely unknown merchants.

`tests/test_scoring.py`

tests/test_scoring.py

TESTS

Automated tests that verify the pressure scoring logic is always correct. Run with: `pytest tests/`

REAL-WORLD ANALOGY

Tests are like quality control checks on a production line. Every time you make a change to the scoring code, you run the tests. If they all pass (green), you know you haven't broken anything. If one fails (red), you know exactly which scenario is now producing wrong results before it reaches real users.

```
import pytest
```

The testing framework. It finds all functions whose names start with `test_` and all classes whose names start with `Test`, runs them, and reports results. You never call these functions yourself.

def make_task(overrides)**

A helper function that creates a valid `TaskBubble` with sensible defaults. The `**overrides` syntax means you can pass any keyword argument to override specific fields. This avoids repeating all 12 required fields in every test. For example: `make_task(importance=10)` creates a bubble with all defaults but importance set to 10.

class TestDeadlineWeight

Each method in this class tests one specific scenario for `_deadline_weight()`. The class groups related tests together so the output is organized. Here's what each test does:

```
test_no_due_date_returns_zero
```

`assert _deadline_weight(None) == 0.0` — if there's no deadline, there's no deadline-based urgency. The `assert` statement checks that the expression is `True`. If it's `False`, `pytest` marks this test as `FAILED`.

```
test_overdue_returns_ten
```

Creates a `datetime` in the past (yesterday) and asserts the function returns `10.0` — maximum urgency.

```
test_naive_datetime_is_handled
```

A "naive" datetime is one without timezone information. The scoring function adds UTC timezone info if missing. This test verifies that doesn't crash.

class TestCalculatePressureScore — Integration tests

`test_score_is_clamped_to_ten`

Creates the worst-case task: overdue, maximum importance, many dependencies, and neglected for 20 days. Without clamping, the raw score would be $10 + 5 + 2 + 3 = 20$. The test verifies the result is exactly 10.0, not 20.

`test_no_pressure_on_fresh_low_importance_task`

A brand new task (no deadline, no dependencies, just created, importance=1). The only contributing factor is importance_weight: $1 \div 2 = 0.5$. The test verifies the score is exactly 0.5. This is a precise mathematical assertion.

`test_dependency_count_default_is_zero`

Verifies that calling `calculate_pressure_score(task)` (without specifying `dependency_count`) gives the same result as calling it with `dependency_count=0` explicitly. Tests that the default parameter works correctly.

Why test private functions like `_deadline_weight`?

Unit tests for private helper functions give you precise control. If `calculate_pressure_score` returns a wrong number, a test against `_deadline_weight` immediately tells you whether the problem is in the deadline component or elsewhere. Without testing each part separately, you'd have to guess which component is broken.

HOW TO RUN THE TESTS

From the `orbi-backend` directory, run: `pytest tests/` — `pytest` finds all test files automatically and runs every test function. You'll see a line per test (green dot = passed, red F = failed) and a summary at the end.

Orbi

Phase 1 Code Study Guide · Generated April 2026

[Open in Chrome or Edge](#) → [Print](#) → [Save as PDF](#)